# Caching

- Improve read performance.

Why needed?

Rahul - eats sweets

- Morning (30 min) → SWEET HOUSE (30 min)
- Afternoon →
- Evening (30 min) →

(90 min wasted in travelling)

SIMRAN —suggests→ buy sweets in morning for afternoon & evening.

↓

Saves Rahul
- 1 hr / day
- 365 hr / year

Caching

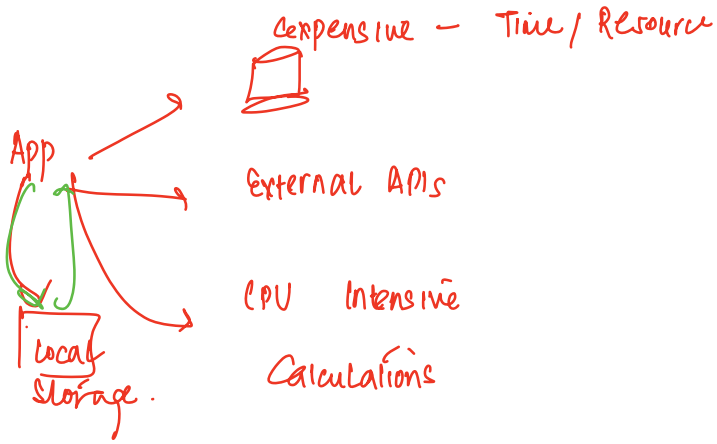
↓

Don't do something repeatedly which doesn't change.

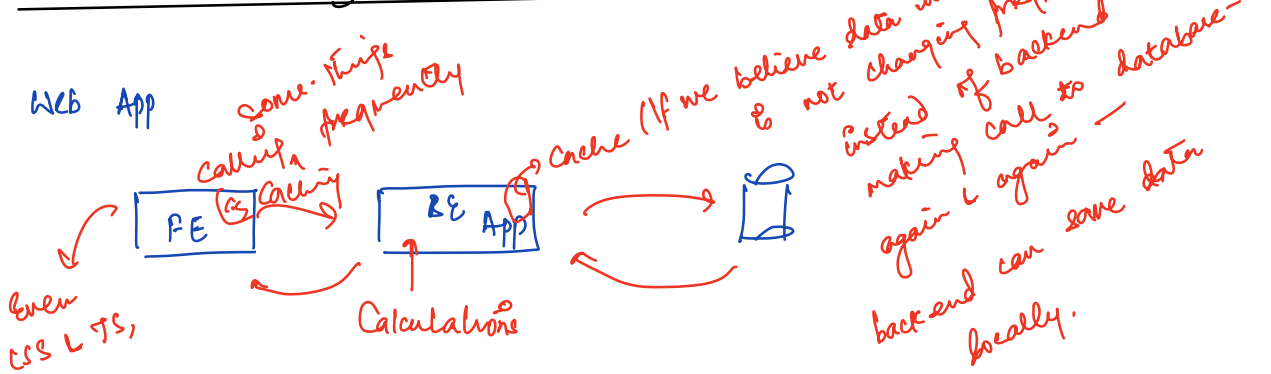
↳ Avoiding unnecessary
- N/W I/Os
- Desc I/Os
- Calculations
- API Calls

→ Being in local memory.

→ Better application performance.




## Places where Caching can be used –




[Many softwares which provide this connection have this inbuilt caching available]

⇓

Similarly from BROWSER – Knows frontend content is not changing frequently – can have caching here also.

## Two types of Cache

① Application Server Cache

(Above explanation)

② CDN (Content Delivery N/w)

eg frontend - CSS, Javascript, images, static content can give to CDN instead of you serving from backend - so searching for data is much lighter.

Caching static content and then delivery to frontend

## Types of Caching Solutions

Local / In-memory Cache

→ Data is stored in local RAM

→ Google Gauva Cache (powerful in-memory caching utility for Java applications)

→ Mem cache → separate caching server (not inside app like Gauva) (key-value store)

↳ distributed in-memory caching system used to speed up dynamic applications by storing data in RAM.

Distributed Cache

External caching (Redis)

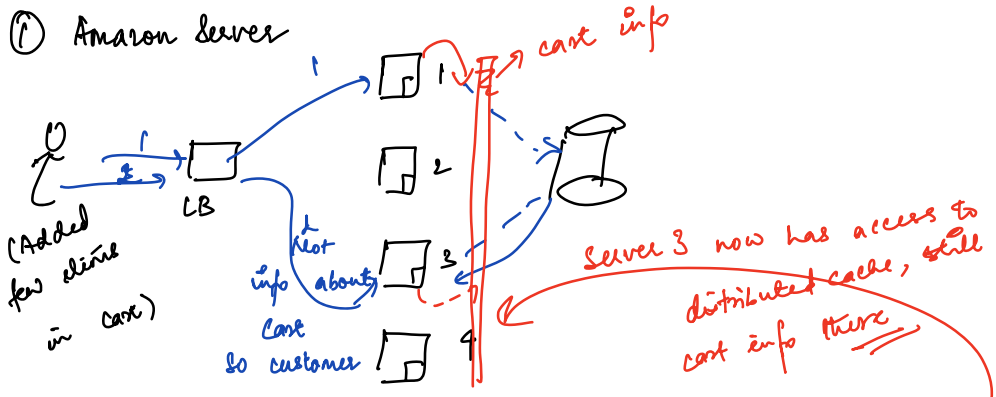
→

distributed cache

They want to share cache.

## Scenarios

① Amazon Server




so customer won't see anything in cart for sometime - bad customer experience so, we'll use distributed cache caching.

## When to use cache?

(a) If data not changing frequently

(If it is - data'll go stale very fast - no use)

(b) Kind of app?

Read - Intensive

(Read > Write)

eg Wikipedia / Newspaper / Blog

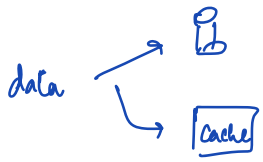
→ Caching makes more sense here.

Write - Intensive → If caching used here - bring down performance as cache has overhead

→ eg Inserting user click info from website

# Different Caching Strategies

## ① Write Through Cache

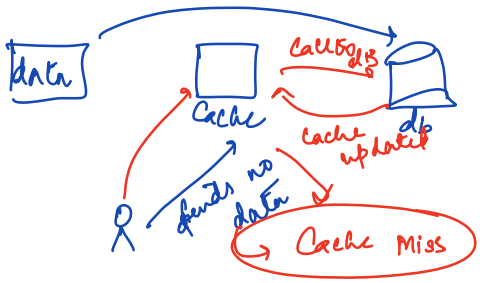



When data comes – write to cache as soon as it is written to database

+ves
- No stale data
- ↑ consistency
- consistent Read

-ve → Slower write (for write-intensive app – very slow)

## ② Write around Cache

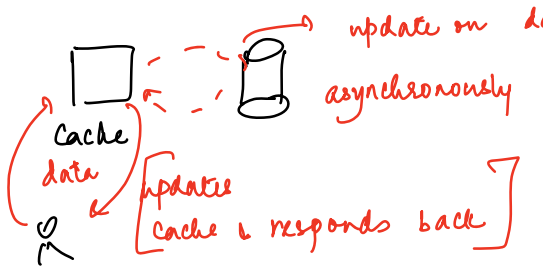



+ves
→ Fast write (write-intensive)

eg Twitter

-ves
→ Read slower → because of cache miss.

## ③ Write Back Cache




+ves → Write fast (Write-intensive)

Challenges → Can lead to loss of data in case of cache memory is failed

# Cache Eviction Strategies

Read limited memory.

- Add new data
- Remove stale data

eg Rahul buys sweets for next 3 days & has no fridge → and after 1 day - that sweet becomes stale - so has to throw it.

eg Rahul brought 9 Rasagullas for 3 days but somebody brings Rasamalai so Rahul has to make space for it. → Eviction strategy

1. First In first Out

   1000 entries

   Cache → New data → first data evicted.

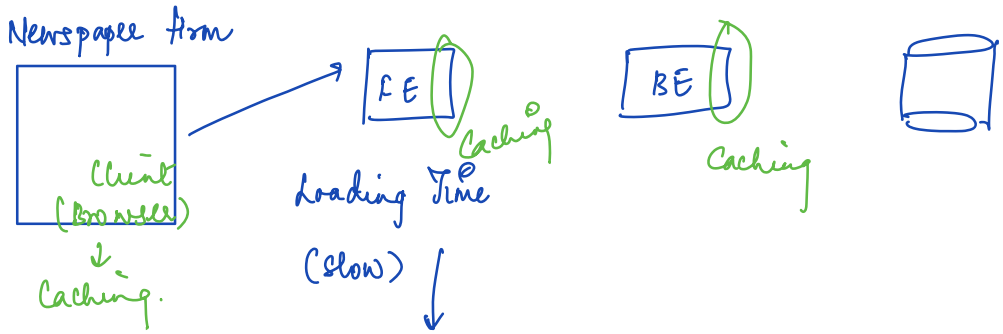
2. Last In first Out

3. LRU → Least Recently Used (Most commonly used)

4. MRU → Most Recently Used

5. RR → Random Replacement

Example




1. Code is already optimised
2. Ready - Heavy App? → Can use caching.

CDN. (JS, CSS, HTML)